Manual técnico y referencia de API

DENTALIS

Sistema de gestión odontológica

Desarrollo y mantenimiento

Alumno: Ledesma Antonella

Materia: Proyecto, Diseño e Implementación de Sistemas Computacionales

2026

Manual técnico y referencia de API

Arquitectura, módulos, autorización, configuración y rutas observadas en el código fuente de DENTALIS. Este documento fue preparado a partir de la inspección del código fuente y la configuración disponible en este proyecto.

Arquitectura

El proyecto se compone de un cliente React Native con Expo, una API HTTP en Express y una base de datos MariaDB/MySQL. El cliente organiza las pantallas con React Navigation y selecciona un stack según el rol de la sesión. Axios envía solicitudes al backend; el middleware JWT protege las rutas privadas (ver Tabla 1).

Tabla 1

Capas de la arquitectura

Capa	Ubicación	Responsabilidad
Cliente Expo	frontend/src/screens, navigation, services	Interfaz móvil, rutas por rol y consumo REST.
API Express	backend/server.js, src/routes, controllers	Autenticación, validaciones, permisos y lógica de aplicación.
Persistencia	backend/src/config/schema.sql, models	Modelo relacional; acceso parametrizado vía mysql2.
Seguridad transversal	middlewares, utils/jwt.js, secureStorage	Autenticación Bearer, verificación de roles, tokens y almacenamiento del cliente.

Módulos de negocio

- **Identidad:** login local, verificación de dos factores, renovación de token, cierre de sesión y OAuth.
- **Clínica:** pacientes, agenda, fichas clínicas, consultas, odontograma y tratamientos.
- **Administración:** usuarios, catálogo, cuentas, pagos, ventas, métodos de pago e informes.
- **Perfil:** consulta de identidad y cambio de contraseña.

Modelo de datos

La Tabla 2 describe las tablas principales del esquema relacional.

Tabla 2

Tablas del modelo de datos

Tabla	Uso principal
usuarios	Cuentas, rol, estado, credenciales cifradas/hash, estado de sesión y datos de 2FA/OAuth.
pacientes	Identidad y datos administrativos del paciente; estado activo/inactivo.
turnos	Paciente, odontólogo, fecha, hora, duración, motivo, notas y estado.
fichas_clinicas	Antecedentes, alergias, medicación, observaciones y odontograma JSON.
consultas	Evolución clínica asociada a ficha y odontólogo.
catalogo_tratamientos	Catálogo administrado con categoría, descripción, precio y duración.
tratamientos	Plan individual asociado a paciente y odontólogo, costo y estado.
pagos	Importes, método, paciente, tratamiento y usuario que registra.
registro_actividad	Eventos administrativos/clínicos auditados por el backend.

Autenticación y autorización

La API emite JWT de acceso y refresh. Las rutas protegidas esperan el encabezado Authorization: Bearer <token>. El middleware authenticate verifica la firma y la expiración, y el middleware authorize compara el rol. El acceso a datos clínicos se restringe al rol odontólogo y, en el controlador, se comprueba la asignación del paciente por agenda donde corresponde (ver Tabla 3).

Tabla 3

Permisos funcionales por rol

Rol	Permisos funcionales principales
administrador	Gestión global y reportes. Administra usuarios y catálogo. Puede crear, editar y cancelar turnos; administra pacientes y pagos.
recepcion	Consulta/creación de pacientes, consulta de agenda, creación de turnos, cambio de estado y pagos.
odontologo	Consulta de agenda y pacientes asignados; edita ficha/odontograma, registra consultas y gestiona tratamientos.
usuario	Solo perfil, cambio de contraseña y ayuda desde la app; sin rutas clínicas asignadas.

API REST

Todas las rutas se publican bajo /api. Salvo login, renovación y OAuth, se requiere autenticación. Las acciones reflejan los permisos definidos actualmente en los routers (ver Tabla 4).

Tabla 4

Rutas de la API y alcance por método

Ruta	Métodos / alcance
/auth	POST /login, /verify-2fa, /refresh, /logout, /oauth.
/users	GET /me; PUT /me/password. Admin: GET/POST /, GET/PATCH /:id. Admin/recepción: GET /odontologos.
/patients	GET / y /:id (admin, recepción, odontólogo); POST/PUT (admin, recepción); DELETE lógico (admin).
/appointments	GET / y /:id (admin, recepción, odontólogo); POST (admin, recepción); PUT (admin); PATCH /:id/estado (los tres perfiles); DELETE (admin).
/clinical-records	GET/PUT /:pacienteId; POST /:pacienteId/consultas; PUT /:pacienteId/odontograma (odontólogo).
/treatments	GET (admin, recepción, odontólogo); POST/PUT/DELETE (odontólogo).
/treatment-catalog	GET (admin, odontólogo); POST/PATCH (admin).
/payments	GET /, /summary, /by-odontologo/:id (admin/recepción); GET /admin-summary (admin); POST / (admin/recepción).

Convenciones de datos

- Fechas de turnos: YYYY-MM-DD. Horas: HH:MM.
- Estados de turno admitidos por validación de actualización: pendiente, confirmado, cancelado, reprogramado y realizado.
- Estados del tratamiento: planificado, en_curso, completado y cancelado.
- Métodos de pago: efectivo, tarjeta_debito, tarjeta_credito, transferencia y obra_social.
- Nueva contraseña: mínimo ocho caracteres. El cambio requiere la contraseña actual.
- Las respuestas de error pasan por un middleware común; en desarrollo se agrega el stack trace.

Configuración de entorno

En el backend se definen PORT, NODE_ENV, DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, JWT_SECRET, JWT_EXPIRES_IN, JWT_REFRESH_SECRET, JWT_REFRESH_EXPIRES_IN y las variables EMAIL/OAuth según las integraciones habilitadas. En el cliente se definen EXPO_PUBLIC_API_URL_WEB, EXPO_PUBLIC_API_URL_NATIVE y los client IDs y callbacks públicos por plataforma. No se deben incluir secretos de servidor en EXPO_PUBLIC_* ni en el bundle móvil.

Mantenimiento

- Aplicar cambios del modelo mediante schema.sql y migraciones versionadas; respaldar antes de migrar.
- Revisar los logs HTTP de Morgan y registro_actividad ante incidentes.
- Mantener secretos en el entorno de despliegue, fuera del repositorio.
- Documentar nuevas rutas con método, rol permitido, validaciones y efectos sobre datos.
- Las rutas de login social dependen de credenciales y callbacks configurados por el operador.

Alcance y observaciones del análisis

El proyecto no incluye una suite de pruebas declarada en backend/package.json; el script test indica que no hay pruebas configuradas. La evidencia de este manual es la lectura estática de código y configuración, no una auditoría de seguridad ni una certificación de despliegue. Se debe revisar siempre schema.sql y los controladores de la versión que se vaya a publicar.

Nota. La API implementa DELETE lógico para pacientes; los turnos usan la cancelación como estado u operación administrativa. Antes de integrar un cliente externo, confirmar campos y respuestas en el controlador específico.